



João Pedro de Sousa Moura

Rio de Janeiro-RJ

2024

Teórica 1. Características de dados

streaming

Dados streaming são gerados continuamente e chegam em tempo real. Eles vêm de fontes como sensores ou logs, e são processados rapidamente para obter insights quase instantâneos.

2. Diferença entre processamento em lote e ingestão de dados

Processamento em lote é a análise de dados acumulados ao longo do tempo. **Ingestão de dados** é o ato de trazer dados para um sistema, podendo ser em tempo real (streaming) ou em lote.

3. Cinco conceitos fundamentais do Apache Kafka •

Produtores: Envia mensagens para o Kafka.

- **Consumidores:** Lêem essas mensagens.
- **Tópicos:** Organizadores das mensagens.
- **Partições:** Dividem tópicos para escalabilidade.
- **Clusters:** Conjunto de brokers que gerenciam e replicam as mensagens.

4. Arquitetura do Apache Kafka

A arquitetura do Kafka inclui **brokers** que armazenam mensagens, **tópicos** organizando-as, e **partições** para facilitar o paralelismo. O **Zookeeper** coordena tudo para que funcione de forma sincronizada.

5. Confiabilidade e tolerância a falhas no Apache Kafka

O Kafka replica partições entre brokers para que, se um falhar, outro assuma. Além disso, ele permite configurações de entrega para garantir que as mensagens sejam entregues pelo menos uma vez.

6. Exemplos de Kafka com NoSQL e SQL

- **NoSQL:** O Kafka pode captar mudanças no MongoDB em tempo real para análise.

- **SQL:** Pode captar transações financeiras de um PostgreSQL e enviá-las a um sistema de detecção de fraudes.

7. Benefícios do Apache Kafka

O Kafka é rápido, escalável e confiável, com uma arquitetura robusta para streaming e pipelines de dados em tempo real.

8. Empresas com Kafka em nuvem

- **Confluent Cloud:** Criado pelos fundadores do Kafka, oferece Kafka como serviço com recursos de análise avançados.
- **AWS MSK:** Serviço gerenciado de Kafka na AWS, que se integra com outros serviços da AWS, como S3 e Lambda.

Prática

1/2)

```

quantidade_total_por_categoria = spark.sql("""
    SELECT Categoria, SUM(Quantidade) AS Quantidade_Total
    FROM vendas
    GROUP BY Categoria
""")
quantidade_total_por_categoria.show()

+-----+-----+
| Categoria|Quantidade_Total|
+-----+-----+
| Categoria 1|          36|
| Categoria 2|          16|
| Categoria 3|           6|
+-----+-----+

[4] # Calculando a receita total por categoria
receita_total_por_categoria = spark.sql("""
    SELECT Categoria, SUM(Quantidade * Preço) AS Receita_Total
    FROM vendas
    GROUP BY Categoria
""")
receita_total_por_categoria.show()

+-----+-----+
| Categoria|Receita_Total|
+-----+-----+
| Categoria 1|        650.0|
| Categoria 2|        870.0|
| Categoria 3|        640.0|
+-----+-----+

```

3/ 4)

```

media_preco_por_categoria = spark.sql("""
    SELECT Categoria, AVG(Preço) AS Preço_Medio
    FROM vendas
    GROUP BY Categoria
""")
media_preco_por_categoria.show()

+-----+-----+
| Categoria|Preço_Medio|
+-----+-----+
| Categoria 1|        17.5|
| Categoria 2|53.33333333333333|
| Categoria 3|106.66666666666667|
+-----+-----+

[6] quantidade_media_por_produto = spark.sql("""
    SELECT Produto, AVG(Quantidade) AS Quantidade_Media
    FROM vendas
    GROUP BY Produto
""")
quantidade_media_por_produto.show()

+-----+-----+
| Produto|Quantidade_Media|
+-----+-----+
| Produto B|           4.5|
| Produto E|           7.0|
| Produto D|           2.0|
| Produto C|           7.0|
| Produto A|          11.0|
| Produto F|           2.0|
+-----+-----+

```

5/ 6)

```
maior_valor_venda = spark.sql("""
SELECT MAX(Quantidade * Preço) AS Maior_Valor_Venda
FROM vendas
""")
maior_valor_venda.show()

+-----+
|Maior_Valor_Venda|
+-----+
|          420.0|
+-----+

[8] produto_mais_vendido = spark.sql("""
SELECT Produto, SUM(Quantidade) AS Quantidade_Total
FROM vendas
GROUP BY Produto
ORDER BY Quantidade_Total DESC
LIMIT 1
""")
produto_mais_vendido.show()

+-----+-----+
| Produto|Quantidade_Total|
+-----+-----+
|Produto A|             22|
+-----+-----+
```

7/ 8)

```
receita_total_por_produto = spark.sql("""
SELECT Produto, SUM(Quantidade * Preço) AS Receita_Total
FROM vendas
GROUP BY Produto
""")
receita_total_por_produto.show()

+-----+-----+
| Produto|Receita_Total|
+-----+-----+
|Produto B|       450.0|
|Produto E|       420.0|
|Produto D|       400.0|
|Produto C|       210.0|
|Produto A|       440.0|
|Produto F|       240.0|
+-----+-----+

[10] vendas_por_mes = spark.sql("""
SELECT MONTH(Data) AS Mes, COUNT(*) AS Numero_Vendas
FROM vendas
GROUP BY MONTH(Data)
ORDER BY Mes
""")
vendas_por_mes.show()

+-----+-----+
|Mes|Numero_Vendas|
+-----+-----+
|  1|           10|
+-----+-----+
```

Referente ao TP2

1/2

```
[16] !./kafka 2.13-3.8.1/bin/zookeeper-server-start.sh -daemon ./kafka 2.13-3.8.1/config/zookeeper.properties
!./kafka 2.13-3.8.1/bin/kafka-server-start.sh -daemon ./kafka 2.13-3.8.1/config/server.properties
lecho "Aguardando 10 segundos até que os serviços kafka e zookeeper estejam funcionando"
!sleep 10

Aguardando 10 segundos até que os serviços kafka e zookeeper estejam funcionando

# Criar o tópico de treino
!./kafka 2.13-3.8.1/bin/kafka-topics.sh --create --topic reco-train2 --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1

# Criar o tópico de teste
!./kafka 2.13-3.8.1/bin/kafka-topics.sh --create --topic reco-test2 --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1

# Listar os tópicos para verificar se foram criados corretamente
!./kafka 2.13-3.8.1/bin/kafka-topics.sh --list --bootstrap-server localhost:9092

Created topic reco-train2.
Created topic reco-test2.
reco-test
reco-test2
reco-train
reco-train2
```

3/4

```

# Instalar a API do Kaggle
!pip install kaggle

# Configurar a chave API do Kaggle
import os
os.environ["KAGGLE_CONFIG_DIR"] = "~/path/to/your/.kaggle"

# dataset Titanic do Kaggle
!kaggle datasets download -d heptapod/titanic

# Descompactar o arquivo ZIP baixado
!unzip titanic.zip

# Listar os arquivos descompactados para verificar o nome correto
ls

# Carregar o dataset em um DataFrame com pandas
import pandas as pd
df = pd.read_csv("train_and_test2.csv")

# Mostrar as primeiras linhas do DataFrame
print(df.head())

```

Requirement already satisfied: kaggle in /usr/local/lib/python3.10/dist-packages (1.6.17)
Requirement already satisfied: six<1.10 in /usr/local/lib/python3.10/dist-packages (from kaggle) (1.16.0)
Requirement already satisfied: certifi<2021.7.22 in /usr/local/lib/python3.10/dist-packages (from kaggle) (2024.8.30)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (from kaggle) (2.8.2)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from kaggle) (2.32.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from kaggle) (4.66.6)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.10/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.10/dist-packages (from kaggle) (2.2.3)
Requirement already satisfied: bleach in /usr/local/lib/python3.10/dist-packages (from kaggle) (6.2.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.10/dist-packages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: text-unidecode<1.3 in /usr/local/lib/python3.10/dist-packages (from python-slugify->kaggle) (1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->kaggle) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->kaggle) (3.10)
Dataset URL: <https://www.kaggle.com/datasets/heptapod/titanic>

```

Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from kaggle) (4.66.6)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.10/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.10/dist-packages (from kaggle) (2.2.3)
Requirement already satisfied: bleach in /usr/local/lib/python3.10/dist-packages (from kaggle) (6.2.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.10/dist-packages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: text-unidecode<1.3 in /usr/local/lib/python3.10/dist-packages (from python-slugify->kaggle) (1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->kaggle) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->kaggle) (3.10)
Dataset URL: https://www.kaggle.com/datasets/heptapod/titanic
license(s): DBCL-1.0
titanic.zip: Skipping, found more recently modified local copy (use --force to force download)
Archive: titanic.zip
replace train_and_test2.csv? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: train_and_test2.csv
kafka 2.13-3.8.1 ratings.csv titanic.zip train_and_test3.csv
kafka 2.13-3.8.1.tgz sample_data train_and_test2.csv
PassengerId  Age   Fare  Sex  SibSp  zero zero.1 zero.2 zero.3 \
0           1  22.0  7.2500  0     1     0     0     0     0
1           2  38.0  71.2833  1     1     0     0     0     0
2           3  26.0  7.9250  1     0     0     0     0     0
3           4  35.0  53.1000  1     1     0     0     0     0
4           5  35.0  8.0500  0     0     0     0     0     0

zero.4 ... zero.12 zero.13 zero.14 Pclass zero.15 zero.16 Embarked \
0     0 ...      0      0      0      3     0     0      2.0
1     0 ...      0      0      0      1     0     0      0.0
2     0 ...      0      0      0      3     0     0      2.0
3     0 ...      0      0      0      1     0     0      2.0
4     0 ...      0      0      0      3     0     0      2.0

zero.17 zero.18 Survived
0     0      0     0
1     0      0     1
2     0      0     1
3     0      0     1
4     0      0     0

[5 rows x 28 columns]

```

```
[21] !ls
```

```

kafka 2.13-3.8.1 ratings.csv titanic.zip train_and_test3.csv
kafka 2.13-3.8.1.tgz sample_data train_and_test2.csv

```

5/6

```

from kafka import KafkaProducer

import pandas as pd
import json

# Carregar o dataset
df = pd.read_csv("train_and_test2.csv")

# Separar as variáveis independentes (features) e a variável dependente (target)
X = df.drop("Survived", axis=1)
y = df["Survived"]

# Dividir o dataset em treino (80%) e teste (20%)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Função para garantir que os dados sejam serializáveis em JSON
def serialize_data(data):
    # Converte os tipos pandas para tipos nativos do Python
    serialized_data = {}
    for key, value in data.items():
        if isinstance(value, pd.Timestamp):
            serialized_data[key] = value.isoformat()
        elif isinstance(value, (int, pd.Int64Dtype)):
            serialized_data[key] = int(value)
        elif isinstance(value, (float, pd.Float64Dtype)):
            serialized_data[key] = float(value)
        else:
            serialized_data[key] = value
    return serialized_data

# Configuração do Kafka
bootstrap_servers = 'localhost:9092'
train_topic = 'train-topic'
test_topic = 'test-topic'

# Criar o produtor Kafka
producer = KafkaProducer(
    bootstrap_servers=bootstrap_servers,
    value_serializer=lambda x: json.dumps(x).encode('utf-8') # Serializar dados para JSON
)

# Enviar dados de treinamento para o Kafka
for index, row in X_train.iterrows():
    data

```

7/8/9

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, col, split

# Inicializar SparkSession
spark = SparkSession.builder \
    .appName("IntegracaoKafka") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.0") \
    .getOrCreate()

# Imprimir mensagem indicando que a sessão foi criada
print("Sessão Spark criada com sucesso!")
```

Sessão Spark criada com sucesso!

```
[40] # Configuração do Kafka
nome_topico_kafka = "reco-train"
servidores_bootstrap_kafka = 'localhost:9092'

# Ler o stream do Kafka
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", servidores_bootstrap_kafka) \
    .option("subscribe", nome_topico_kafka) \
    .option("startingOffsets", "earliest") \
    .load()

# Imprimir mensagem indicando que o stream foi definido
print("Stream de dados do Kafka foi definido com sucesso!")
```

Stream de dados do Kafka foi definido com sucesso!

10

```
# Sessão Spark criada com sucesso!
```

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, col, split

# Inicializar SparkSession
spark = SparkSession.builder \
    .appName("IntegracaoKafka") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.0") \
    .getOrCreate()

# Imprimir mensagem indicando que a sessão foi criada
print("Sessão Spark criada com sucesso!")
```

```
[40] # Configuração do Kafka
nome_topico_kafka = "reco-train"
servidores_bootstrap_kafka = 'localhost:9092'

# Ler o stream do Kafka
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", servidores_bootstrap_kafka) \
    .option("subscribe", nome_topico_kafka) \
    .option("startingOffsets", "earliest") \
    .load()

# Imprimir mensagem indicando que o stream foi definido
print("Stream de dados do Kafka foi definido com sucesso!")
```

Stream de dados do Kafka foi definido com sucesso!

11/12

Referente ao TP3

1)

```
[1] # Instalando a biblioteca kafka-python
pip install kafka-python -q

246.5/246.5 KB 4.3 MB/s eta 0:00:00

[2] import os
from datetime import datetime
import time
import threading
import json
from json import loads
from csv import DictReader
from kafka import KafkaProducer
from kafka.errors import KafkaError
import pandas as pd
from pandas import json_normalize
from sklearn.model_selection import train_test_split

[3] !curl -sO https://downloads.apache.org/kafka/3.8.1/kafka-2.13-3.8.1.tgz
!tar -xzf kafka-2.13-3.8.1.tgz

[4] !./kafka-2.13-3.8.1/bin/zookeeper-server-start.sh -daemon ./kafka-2.13-3.8.1/config/zookeeper.properties
!./kafka-2.13-3.8.1/bin/kafka-server-start.sh -daemon ./kafka-2.13-3.8.1/config/server.properties
!echo "Aguardando 20 segundos até que os serviços kafka e zookeeper estejam funcionando"
!sleep 20

Aguardando 20 segundos até que os serviços kafka e zookeeper estejam funcionando

[5] !./kafka-2.13-3.8.1/bin/kafka-topics.sh --create --bootstrap-server 127.0.0.1:9092 --replication-factor 1 --partitions 1 --topic reco-train
!./kafka-2.13-3.8.1/bin/kafka-topics.sh --create --bootstrap-server 127.0.0.1:9092 --replication-factor 1 --partitions 1 --topic reco-test
!echo "Aguardando 10 segundos até que os tópicos sejam criados"
!sleep 10

Created topic reco-train.
Created topic reco-test.
Aguardando 10 segundos até que os tópicos sejam criados
```

```
!./kafka-2.13-3.8.1/bin/kafka-topics.sh --describe --bootstrap-server 127.0.0.1:9092 --topic reco-train
!./kafka-2.13-3.8.1/bin/kafka-topics.sh --describe --bootstrap-server 127.0.0.1:9092 --topic reco-test

[2024-11-26 02:17:15,716] WARN [AdminClient clientId=adminclient-1] The DescribeTopicPartitions API is not supported, using Metadata API to describe topics. (org.apache.kafka.clients.admin.KafkaAdminClient)
Topic: reco-train TopicId: h3uafkjj1aaag3lZ_25w PartitionCount: 1 ReplicationFactor: 1 Configs:
Topic: reco-train Partitions: 0 Leader: 0 Replicas: 0 Isr: 0 Elr: N/A LastKnownElr: N/A
[2024-11-26 02:17:18,222] WARN [AdminClient clientId=adminclient-1] The DescribeTopicPartitions API is not supported, using Metadata API to describe topics. (org.apache.kafka.clients.admin.KafkaAdminClient)
Topic: reco-test TopicId: iXwJcRk]w-7NhuRAbCzFA PartitionCount: 1 ReplicationFactor: 1 Configs:
Topic: reco-test Partitions: 0 Leader: 0 Replicas: 0 Isr: 0 Elr: N/A LastKnownElr: N/A

[9] # URL do dataset de voos (Openflights)
url = "https://raw.githubusercontent.com/jpatokal/openflights/master/data/airports.dat"

# Nome do arquivo para salvar localmente
output_file = "airports.csv"

# Download do dataset
!wget --quiet --show-progress (url) -O (output_file)

# Carregar o arquivo com pandas
import pandas as pd

column_names = [
    "Airport_ID", "Name", "City", "Country", "IATA", "ICAO", "Latitude",
    "Longitude", "Altitude", "Timezone", "DST", "Tz_database_time_zone", "Type", "Source"
]

df = pd.read_csv(output_file, header=None, names=column_names)

# Mostrar as 5 primeiras linhas do dataset
print(df.head())

airports.csv 100%[=====] 1.07M ---KB/s in 0.06s
Airport_ID Name City \
0 1 Goroka Airport Goroka
1 2 Madang Airport Madang
2 3 Mount Hagen Kagamuga Airport Mount Hagen
3 4 Nadzab Airport Nadzab
4 5 Port Moresby Jacksons International Airport Port Moresby

Country IATA ICAO Latitude Longitude Altitude Timezone DST \
0 Papua New Guinea GKA AYGA -6.081690 145.391998 5282 10 U
1 Papua New Guinea MAG AYMD -5.287080 145.789001 20 10 U
2 Papua New Guinea HGU AYWH -5.826700 144.296005 5388 10 U
3 Papua New Guinea LAE AYNZ -6.560883 146.725977 239 10 U
4 Papua New Guinea POM AYPY -9.443380 147.220001 146 10 U

Tz_database_time_zone Type Source
0 Pacific/Port_Moresby airport OurAirports
1 Pacific/Port_Moresby airport OurAirports
2 Pacific/Port_Moresby airport OurAirports
3 Pacific/Port_Moresby airport OurAirports
4 Pacific/Port_Moresby airport OurAirports
```

```
airports.csv 100%[=====] 1.07M ---KB/s in 0.06s
Airport_ID Name City \
0 1 Goroka Airport Goroka
1 2 Madang Airport Madang
2 3 Mount Hagen Kagamuga Airport Mount Hagen
3 4 Nadzab Airport Nadzab
4 5 Port Moresby Jacksons International Airport Port Moresby

Country IATA ICAO Latitude Longitude Altitude Timezone DST \
0 Papua New Guinea GKA AYGA -6.081690 145.391998 5282 10 U
1 Papua New Guinea MAG AYMD -5.287080 145.789001 20 10 U
2 Papua New Guinea HGU AYWH -5.826700 144.296005 5388 10 U
3 Papua New Guinea LAE AYNZ -6.560883 146.725977 239 10 U
4 Papua New Guinea POM AYPY -9.443380 147.220001 146 10 U

Tz_database_time_zone Type Source
0 Pacific/Port_Moresby airport OurAirports
1 Pacific/Port_Moresby airport OurAirports
2 Pacific/Port_Moresby airport OurAirports
3 Pacific/Port_Moresby airport OurAirports
4 Pacific/Port_Moresby airport OurAirports

[15] train_df, test_df = train_test_split(df, test_size=0.3, shuffle=True)
print("Número de amostras de treino: ",len(train_df))
print("Número de amostras de teste: ",len(test_df))

Número de amostras de treino: 5388
Número de amostras de teste: 2310

[16] #Inicializar o cliente Kafka Producer
producer = KafkaProducer(bootstrap_servers=['localhost:9092'])

print(f'Iniciando o Kafka producer at {datetime.utcnow()}')

Iniciando o Kafka producer at 2024-11-26 02:30:39.517449
```

```
from kafka import KafkaProducer
import json

topic_name = 'data-topic_train'
counter = 0

# Converte o DataFrame para dicionários e envia cada linha como mensagem
for index, row in train_df.iterrows():
    # Converte a linha para dicionário
    row_dict = row.to_dict()
    key = str(counter).encode('utf-8')

    # Envia a mensagem para o tópico Kafka
    ack = producer.send(topic=topic_name, key=key, value=json.dumps(row_dict).encode('utf-8'))
    metadata = ack.get() # Aguarda o envio da mensagem e captura os metadados

    # Incrementa o contador
    counter += 1
    print(metadata.topic, metadata.partition, key)

# Aguarda que todas as mensagens sejam enviadas antes de encerrar
producer.flush()
print('Escrito {} mensagens no tópico: {}'.format(counter, topic_name))
```

```
data-topic_train 0 b'5331'
data-topic_train 0 b'5332'
data-topic_train 0 b'5333'
data-topic_train 0 b'5334'
data-topic_train 0 b'5335'
data-topic_train 0 b'5336'
data-topic_train 0 b'5337'
data-topic_train 0 b'5338'
data-topic_train 0 b'5339'
data-topic_train 0 b'5340'
data-topic_train 0 b'5341'
data-topic_train 0 b'5342'
data-topic_train 0 b'5343'
data-topic_train 0 b'5344'
data-topic_train 0 b'5345'
data-topic_train 0 b'5346'
data-topic_train 0 b'5347'
data-topic_train 0 b'5348'
data-topic_train 0 b'5349'
```

```
from kafka import KafkaProducer
import json

topic_name = 'data-topic_test'
counter = 0

# Converte o DataFrame para dicionários e envia cada linha como mensagem
for index, row in test_df.iterrows():
    # Converte a linha para dicionário
    row_dict = row.to_dict()
    key = str(counter).encode('utf-8')

    # Envia a mensagem para o tópico Kafka
    ack = producer.send(topic=topic_name, key=key, value=json.dumps(row_dict).encode('utf-8'))
    metadata = ack.get() # Aguarda o envio da mensagem e captura os metadados

    # Incrementa o contador
    counter += 1
    print(metadata.topic, metadata.partition, key)

# Aguarda que todas as mensagens sejam enviadas antes de encerrar
producer.flush()
print("Escrito {} mensagens no tópico: {}".format(counter, topic_name))
```

```
data-topic_test 0 b'2253'
data-topic_test 0 b'2254'
data-topic_test 0 b'2255'
data-topic_test 0 b'2256'
data-topic_test 0 b'2257'
data-topic_test 0 b'2258'
data-topic_test 0 b'2259'
data-topic_test 0 b'2260'
data-topic_test 0 b'2261'
data-topic_test 0 b'2262'
data-topic_test 0 b'2263'
data-topic_test 0 b'2264'
data-topic_test 0 b'2265'
data-topic_test 0 b'2266'
data-topic_test 0 b'2267'
data-topic_test 0 b'2268'
data-topic_test 0 b'2269'
data-topic_test 0 b'2270'
```

```
[28] #Crie um consumidor todas as mensagens do tópico, mas não marque como 'lidas' (enable_auto_commit=False)
#para que possamos re-lê-las quantas vezes quisermos.
topic_name_train = 'data-topic_train'

consumer_train = KafkaConsumer(topic_name_train,
                                group_id = 'topicos_train',
                                bootstrap_servers=['localhost:9092'],
                                value_deserializer=lambda m: json.loads(m.decode('utf-8')),
                                auto_offset_reset = 'earliest',
                                enable_auto_commit=False,
                                consumer_timeout_ms = 10000
                                )
```

```
[26] #Visualizar as 5 primeiras linhas
conjunto_train.head(5)
```

	Airport_ID	Name	City	Country	IATA	ICAO	Latitude	Longitude	Altitude	Timezone	DSI	Tr_database_time_zone	Type	Source
0	1295	Montauban Airport	Montauban	France	XMW	LFDB	44.025700	1.378040	351	1	E	Europe/Paris	airport	OurAirports
0	6202	Pakhokku Airport	Pakhokku	Burma	PKK	VYPU	21.404300	95.111250	151	6.5	U	Asia/Rangoon	airport	OurAirports
0	1573	České Budějovice Airport	České Budějovice	Czech Republic	VN	LKCS	48.946400	14.427500	1417	1	E	Europe/Prague	airport	OurAirports
0	8515	Aiken Regional Airport	Aiken	United States	AIK	KAIK	33.649399	-81.684998	528	-5	A	America/New_York	airport	OurAirports
0	12950	Statesboro Bulloch County Airport	Statesboro	United States	TBR	KTBR	32.482700	-81.736900	187	-5	A	VN	airport	OurAirports

Próximas etapas: [Gerar código com conjunto_train](#) [Ver gráficos recomendados](#) [New interactive sheet](#)

```
#Obter informações do Dataframe
conjunto_train.info()

<class 'pandas.core.frame.DataFrame'>
Index: 5388 entries, 0 to 0
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype  
---  --
0   Airport_ID             5388 non-null  int64   
1   Name                   5388 non-null  object  
2   City                   5355 non-null  object  
3   Country                5388 non-null  object  
4   IATA                   5388 non-null  object  
5   ICAO                   5388 non-null  object  
6   Latitude               5388 non-null  float64  
7   Longitude              5388 non-null  float64  
8   Altitude               5388 non-null  int64   
9   Timezone               5388 non-null  object  
10  DST                    5388 non-null  object  
11  Tz_database_time_zone  5388 non-null  object  
12  Type                   5388 non-null  object  
13  Source                 5388 non-null  object  
dtypes: float64(2), int64(2), object(10)
memory usage: 631.44 KB

[28] #Crie um consumidor todas as mensagens do tópico, mas não marque como 'lidas' (enable_auto_commit=False)
#para que possamos relê-los quantas vezes quisermos.
topic_name_test = 'data-topic_test'

consumer_test = KafkaConsumer(topic_name_test,
                              group.id = 'topicos_test',
                              bootstrap_servers=['localhost:9092'],
                              value_deserializer=lambda m: json.loads(m.decode('utf-8'))),
                              auto_offset_reset = 'earliest',
                              enable_auto_commit=False,
                              consumer_timeout_ms = 10000
                              )
```

```
#Ler o consumidor, concatenar todas as linhas, normalizando o json
lista_test = []

for message in consumer_test:
    global combined
    data = pd.json_normalize(message.value)
    lista_test.append(data)

conjunto_test = pd.concat(lista_test)

[30] #Visualizar as 5 primeiras linhas
conjunto_test.head(5)
```

	Airport_ID	Name	City	Country	IATA	ICAO	Latitude	Longitude	Altitude	Timezone	DST	Tz_database_time_zone	Type	Source
0	2299	Asahikawa Airport	Asahikawa	Japan	AKJ	RJEC	43.670799	142.447006	721	9	U	Asia/Tokyo	airport	OurAirports
0	7902	Toronto/Oshawa Executive Airport	Oshawa	Canada	YOO	CYOO	43.922798	-78.894997	460	-5	A	America/Toronto	airport	OurAirports
0	8122	Portland Hillsboro Airport	Hillsboro	United States	HIO	KHIO	45.540401	-122.949997	208	-8	A	America/Los_Angeles	airport	OurAirports
0	8691	Akeno Airport	Akeno	Japan	YN	RJOE	34.533298	136.671997	20	9	U	Asia/Tokyo	airport	OurAirports
0	2733	La Nubia Airport	Manizales	Colombia	MZL	SKMZ	5.029600	-75.464700	6871	-5	U	America/Bogota	airport	OurAirports

Próximas etapas: [Gerar código com conjunto_test](#) [Ver gráficos recomendados](#) [New interactive sheet](#)

```
[31] #Obter informações do Dataframe
conjunto_test.info()

<class 'pandas.core.frame.DataFrame'>
Index: 2310 entries, 0 to 0
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype  
---  --
0   Airport_ID             2310 non-null  int64   
1   Name                   2310 non-null  object  
2   City                   2294 non-null  object  
3   Country                2310 non-null  object  
4   IATA                   2310 non-null  object  
5   ICAO                   2310 non-null  object  
6   Latitude               2310 non-null  float64  
7   Longitude              2310 non-null  float64
```

```
[32] # Instalando o Spark
!pip install pyspark -q

from pyspark.sql import SparkSession

# Inicie uma sessão Spark
spark = SparkSession.builder \
    .appName("MachineLearning") \
    .getOrCreate()

[34] from pyspark.sql import SQLContext

# Carregar dataframes pandas no Spark
sql_context = SQLContext(spark)
df_treino = sql_context.createDataFrame(conjunto_train)
df_teste = sql_context.createDataFrame(conjunto_test)

/usr/local/lib/python3.10/dist-packages/pyspark/sql/context.py:113: FutureWarning: Deprecated in 3.0.0. Use SparkSession.builder.getOrCreate() instead.
warnings.warn()
```

```

# Verifica o dataframe de treino
df_treino.show()

[1]
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Airport_ID | Name | City | Country | IATA | ICAO | Latitude | Longitude | Altitude | Timezone | DST | Tz_database_time_zone | Type | Source |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1295 | Montauban Airport | Montauban | France | MMT | LFDB | 44.02569961547852 | 1.3788399568928345 | 351 | 1 | E | Europe/Paris | airport | OurAirports |
| 6282 | Pakhokku Airport | Pakhokku | Burma | PKK | VYPU | 21.4843 | 95.11125 | 151 | 6.5 | U | Asia/Rangoon | airport | OurAirports |
| 1573 | Ceske Budejovice ... | Ceske Budejovice | Czech Republic | VU | LKCS | 48.9463996887287 | 14.42748977118164 | 1417 | 1 | E | Europe/Prague | airport | OurAirports |
| 8515 | Aliken Regional Ai... | Aliken | United States | AIK | KAIK | 33.649399 | -81.684988 | 528 | -5 | A | America/New_York | airport | OurAirports |
| 12958 | Statesboro Bulloc... | Statesboro | United States | TBN | KTBH | 32.4827803479 | -81.73698832559999 | 187 | -5 | A | Africa/Algiers | airport | OurAirports |
| 2191 | Relizane Airport | Relizane | Algeria | VU | DAAZ | 35.6853881 | -117.6928813 | 2283 | -8 | A | America/Los_Angeles | airport | OurAirports |
| 3490 | China Lake Naws (... | China Lake | United States | EGI | KEG1 | 35.6853881 | -117.6928813 | 2283 | -8 | A | America/Los_Angeles | airport | OurAirports |
| 11833 | Duke Field | Crestview | United States | EGI | KEG1 | 30.65848016 | -86.52298344 | 191 | W | W | America/Los_Angeles | airport | OurAirports |
| 11358 | Palanan Community... | Palanan | Philippines | VU | RPUN | 17.065667 | 122.427671 | 52 | 8 | W | Asia/Makassar | airport | OurAirports |
| 3924 | Muhammad Salahudd... | Bima | Indonesia | BMU | WADB | -8.5396499613789 | 118.6869964596 | 3 | 8 | W | Asia/Makassar | airport | OurAirports |
| 3665 | Waynesville-St. R... | Fort Leonardwood | United States | TBN | KTBH | 37.74160004 | -92.14078129 | 1159 | -6 | A | America/Chicago | airport | OurAirports |
| 3587 | Akron Fulton Inte... | Akron | United States | AKC | KAKR | 41.0374884741 | -81.466983685 | 1067 | -5 | A | America/New_York | airport | OurAirports |
| 7693 | Gillespie Field | El Cajon | United States | SEE | KSEE | 32.826108577881 | -116.9728812287 | 388 | -8 | A | America/Los_Angeles | airport | OurAirports |
| 6223 | Radin Inten II (B... | Bandar Lampung-Su... | Indonesia | TKG | WIAI | -5.240556 | 105.175556 | 282 | 7 | W | Asia/Jakarta | airport | OurAirports |
| 5968 | Gustavus Airport | Gustavus | United States | GSY | PAG5 | 58.4253886 | -135.787887 | 35 | -9 | A | America/Anchorage | airport | OurAirports |
| 5781 | Logroño-Agoncillo... | Logroño-Agoncillo... | Spain | RJL | LLELO | 42.4609534888 | -2.32223518742 | 1161 | 1 | E | Europe/Madrid | airport | OurAirports |
| 1880 | Masvingo Internat... | Masvingo | Zimbabwe | RWZ | FMMV | -20.85529975891133 | 30.859188341796875 | 3595 | 2 | U | Africa/Harare | airport | OurAirports |
| 3375 | Zhengzhou Xinchén... | Zhengzhou | China | CGO | ZKCC | 34.51969986699995 | 113.841803418 | 495 | 8 | U | Asia/Shanghai | airport | OurAirports |
| 1121 | Egal Internationa... | Hargeisa | Somalia | HGA | HCHH | 9.518170356750488 | 44.08879852294922 | 4423 | 3 | U | Africa/Mogadishu | airport | OurAirports |
| 43 | Burwash Airport | Burwash | Canada | YDB | CYDB | 61.37118137939453 | -139.04180836621894 | 2647 | -8 | A | America/Vancouver | airport | OurAirports |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 20 rows

```

```

[36] # Remover linhas com NaN apenas nas colunas 'Latitude' e 'Longitude'
df_clean = df.dropna(subset=["Latitude", "Longitude"])

# Se você tiver outros DataFrames para limpar, aplique o mesmo processo
df_treino_clean = df_treino.dropna(subset=["Latitude", "Longitude"])
df_teste_clean = df_teste.dropna(subset=["Latitude", "Longitude"])

```

```

# Verifica o dataframe de treino
df_treino_clean.show()

[1]
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Airport_ID | Name | City | Country | IATA | ICAO | Latitude | Longitude | Altitude | Timezone | DST | Tz_database_time_zone | Type | Source |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1295 | Montauban Airport | Montauban | France | MMT | LFDB | 44.02569961547852 | 1.3788399568928345 | 351 | 1 | E | Europe/Paris | airport | OurAirports |
| 6282 | Pakhokku Airport | Pakhokku | Burma | PKK | VYPU | 21.4843 | 95.11125 | 151 | 6.5 | U | Asia/Rangoon | airport | OurAirports |
| 1573 | Ceske Budejovice ... | Ceske Budejovice | Czech Republic | VU | LKCS | 48.9463996887287 | 14.42748977118164 | 1417 | 1 | E | Europe/Prague | airport | OurAirports |
| 8515 | Aliken Regional Ai... | Aliken | United States | AIK | KAIK | 33.649399 | -81.684988 | 528 | -5 | A | America/New_York | airport | OurAirports |
| 12958 | Statesboro Bulloc... | Statesboro | United States | TBN | KTBH | 32.4827803479 | -81.73698832559999 | 187 | -5 | A | Africa/Algiers | airport | OurAirports |
| 2191 | Relizane Airport | Relizane | Algeria | VU | DAAZ | 35.752281 | 0.626272 | 282 | 1 | N | Africa/Algiers | airport | OurAirports |
| 3490 | China Lake Naws (... | China Lake | United States | EGI | KEG1 | 35.6853881 | -117.6928813 | 2283 | -8 | A | America/Los_Angeles | airport | OurAirports |
| 11833 | Duke Field | Crestview | United States | EGI | KEG1 | 30.65848016 | -86.52298344 | 191 | W | W | America/Los_Angeles | airport | OurAirports |
| 11358 | Palanan Community... | Palanan | Philippines | VU | RPUN | 17.065667 | 122.427671 | 52 | 8 | W | Asia/Makassar | airport | OurAirports |
| 3924 | Muhammad Salahudd... | Bima | Indonesia | BMU | WADB | -8.5396499613789 | 118.6869964596 | 3 | 8 | W | Asia/Makassar | airport | OurAirports |
| 3665 | Waynesville-St. R... | Fort Leonardwood | United States | TBN | KTBH | 37.74160004 | -92.14078129 | 1159 | -6 | A | America/Chicago | airport | OurAirports |
| 3587 | Akron Fulton Inte... | Akron | United States | AKC | KAKR | 41.0374884741 | -81.466983685 | 1067 | -5 | A | America/New_York | airport | OurAirports |
| 7693 | Gillespie Field | El Cajon | United States | SEE | KSEE | 32.826108577881 | -116.9728812287 | 388 | -8 | A | America/Los_Angeles | airport | OurAirports |
| 6223 | Radin Inten II (B... | Bandar Lampung-Su... | Indonesia | TKG | WIAI | -5.240556 | 105.175556 | 282 | 7 | W | Asia/Jakarta | airport | OurAirports |
| 5968 | Gustavus Airport | Gustavus | United States | GSY | PAG5 | 58.4253886 | -135.787887 | 35 | -9 | A | America/Anchorage | airport | OurAirports |
| 5781 | Logroño-Agoncillo... | Logroño-Agoncillo... | Spain | RJL | LLELO | 42.4609534888 | -2.32223518742 | 1161 | 1 | E | Europe/Madrid | airport | OurAirports |
| 1880 | Masvingo Internat... | Masvingo | Zimbabwe | RWZ | FMMV | -20.85529975891133 | 30.859188341796875 | 3595 | 2 | U | Africa/Harare | airport | OurAirports |
| 3375 | Zhengzhou Xinchén... | Zhengzhou | China | CGO | ZKCC | 34.51969986699995 | 113.841803418 | 495 | 8 | U | Asia/Shanghai | airport | OurAirports |
| 1121 | Egal Internationa... | Hargeisa | Somalia | HGA | HCHH | 9.518170356750488 | 44.08879852294922 | 4423 | 3 | U | Africa/Mogadishu | airport | OurAirports |
| 43 | Burwash Airport | Burwash | Canada | YDB | CYDB | 61.37118137939453 | -139.04180836621894 | 2647 | -8 | A | America/Vancouver | airport | OurAirports |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 20 rows

```

```

from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer

# Selecionando as colunas numéricas para o VectorAssembler
input_columns = ["Latitude", "Longitude", "Altitude"]

# Configurar o VectorAssembler para as colunas de entrada
assembler = VectorAssembler(inputCols=input_columns, outputCol="features", handleInvalid="skip")

# Criar o modelo de Regressão Linear
lr = LinearRegression(featuresCol="features", labelCol="Altitude") # Supondo que 'Altitude' é a variável alvo

# Definindo o Pipeline
pipeline = Pipeline(stages=[assembler, lr])

# Ajustar o modelo com os dados de treino
lr_model = pipeline.fit(df_treino_clean)

# Realizar previsões no conjunto de teste
predictions = lr_model.transform(df_teste_clean)

# Exibir algumas previsões
predictions.select("features", "Altitude", "prediction").show(5)

+-----+-----+-----+
| features | Altitude | prediction |
+-----+-----+-----+
|[43.6707992553710...| 721 | 721.0 |
|[43.922798,-78.89...| 460 | 460.0 |
|[45.540401,-122.9...| 208 | 207.99999999999994 |
|[34.5332984924316...| 20 | 19.999999999999999 |
|[5.0296,-75.4647,...| 6871 | 6871.0000000000002 |
+-----+-----+-----+
only showing top 5 rows

```

```
from pyspark.ml.regression import LinearRegression
from pyspark.ml.feature import VectorAssembler

# Definir as colunas numéricas para a transformação
input_columns = ["Latitude", "Longitude", "Altitude"]

# Configurar o VectorAssembler para as colunas de entrada
assembler = VectorAssembler(inputCols=input_columns, outputCol="features", handleInvalid="skip")

# Transformar os dados de treino com o assembler
train_data = assembler.transform(df_treino_clean)

# Inicializar o modelo de regressão linear com as colunas 'features' e a variável de rótulo 'Altitude'
lr = LinearRegression(featuresCol="features", labelCol="Altitude") # Usando 'Altitude' como variável alvo

# Treinar o modelo
lr_model = lr.fit(train_data)

# Realizar previsões
predictions = lr_model.transform(train_data)

# Exibir as primeiras previsões
predictions.select("features", "Altitude", "prediction").show(5)
```

features	Altitude	prediction
[44.0256996154785...]	351	350.99999999999994
[21.4043, 95.11125...]	151	150.99999999999997
[48.9463996887207...]	1417	1417.0000000000002
[33.649399, -81.68...]	528	528.0
[32.4827003479, -8...]	187	186.99999999999997

only showing top 5 rows

```
from pyspark.ml.evaluation import RegressionEvaluator

# Avaliar o modelo com RMSE e R2, usando 'Altitude' como variável alvo
rmse_evaluator = RegressionEvaluator(labelCol="Altitude", predictionCol="prediction", metricName="rmse")
r2_evaluator = RegressionEvaluator(labelCol="Altitude", predictionCol="prediction", metricName="r2")

# Avaliar as previsões
rmse = rmse_evaluator.evaluate(predictions)
r2 = r2_evaluator.evaluate(predictions)

# Exibir os resultados
print(f"Root Mean Squared Error (RMSE): {rmse}")
print(f"R2: {r2}")
```

Root Mean Squared Error (RMSE): 4.090204523057414e-13
R2: 1.0

valor de R^2 igual a 1.0 sugere que o modelo conseguiu explicar 100% da variação dos dados de treino

Parte 5 Assessment

1. Três conceitos fundamentais do Google BigQuery

Data Warehouse como Serviço:

O BigQuery é um data warehouse que não precisa de servidores ou infraestrutura. É só subir os dados e começar a analisar.

Consulta com SQL:

Você usa SQL padrão para consultar os dados, o que facilita para quem já conhece a linguagem.

Escalabilidade Automática:

Ele ajusta o desempenho automaticamente, mesmo com grandes volumes de dados.

2. Arquitetura do Google BigQuery

O BigQuery tem três partes principais:

Armazenamento: Os dados ficam armazenados de forma otimizada e distribuída.

Processamento: As consultas rodam em paralelo, distribuídas por vários servidores.

Controle: Ele gerencia os recursos, segurança e escalabilidade sem você precisar se preocupar.

3. Hierarquia de Dados no BigQuery

Projetos: A estrutura principal, onde você organiza tudo.

Datasets: Agrupam tabelas dentro do projeto.

Tabelas e Views: Tabelas armazenam dados; views são consultas salvas.

Registros e Campos: São as linhas e colunas dentro das tabelas.

4. Benefícios do Google BigQuery

É super rápido para analisar grandes volumes de dados.

Você só paga pelo que usa, sem gastar à toa.

Não precisa gerenciar servidores, e ele se adapta automaticamente à demanda.

Funciona bem com outras ferramentas do Google.

É seguro e fácil de controlar quem acessa os dados.

Parte 6 Assessment

```
Calcular a soma do campo tripduration agrupado por gênero.

# Definir a consulta SQL para calcular a soma do tripduration por gênero
query_soma = """
SELECT
    gender,
    SUM(tripduration) AS total_tripduration_seconds
FROM
    `bigquery-public-data.new_york_citibike.citibike_trips`
WHERE
    gender IN ('male', 'female')
GROUP BY
    gender
"""

# Executar a consulta e transformar os resultados em um DataFrame
try:
    result_soma = client.query(query_soma).to_dataframe()
    print("Soma do campo tripduration por gênero:")
    print(result_soma)
except Exception as e:
    print(f"Erro ao executar a consulta para soma: {e}")

Erro ao executar a consulta para soma: 400 POST https://bigquery.googleapis.com/bigquery/v2/projects/995558026489/jobs?prettyPrint=false: ProjectId must be non-empty
Location: None
Job ID: bace3220-66d4-4966-9d49-36c67d2f8ed8
```

```
# Definir a consulta SQL para calcular a quantidade de mulheres com tripduration maior que 10 minutos
query_mulheres_10_min = """
SELECT
    COUNTIF(gender = 'female' AND tripduration > 600) AS female_trips_over_10_minutes
FROM
    `bigquery-public-data.new_york_citibike.citibike_trips`
WHERE
    gender = 'female'
"""

# Executar a consulta e transformar os resultados em um DataFrame
try:
    result_mulheres_10_min = client.query(query_mulheres_10_min).to_dataframe()
    print("Quantidade de mulheres com tripduration maior que 10 minutos:")
    print(result_mulheres_10_min)
except Exception as e:
    print(f"Erro ao executar a consulta para mulheres com tripduration > 10 minutos: {e}")

Erro ao executar a consulta para mulheres com tripduration > 10 minutos: 400 POST https://bigquery.googleapis.com/bigquery/v2/projects/995558026489/jobs?prettyPrint=false: ProjectId must be non-empty
Location: None
Job ID: 008697fc-13f5-4928-ab61-995ab73e18a8
```

Professor, não consegui fazer a conexão com o cloud, ficou dando erro repetidamente. fiz a criação da conta, o cadastro, a criação do projeto e não conseguia fazer com que aparecesse no backlog. Só tenho hoje para fazer o projeto pois irei viajar amanhã e o projeto só foi disponibilizado hoje.